

LAB3 - Les fichiers



Table des matières

1 - Objectifs.....	2
2 - Travaux préparatoires.....	2
3 - Manipulation de fichiers.....	2
3.1 - Introduction.....	2
3.2 - Accès aux informations de l'inode.....	3
3.3 - Fonctions de manipulation.....	4
1 Autorisation d'accès.....	5
2 Modification des attributs.....	5
3 Création/suppression.....	6
4 Accès au contenu.....	6

1 Objectifs

Ce LAB présente les notions de base de gestions de fichiers. Il est basé sur la mise en parallèle de notions relatives aux commandes Shell avec leur équivalent en langage C.

- Documents : *Linux – les commandes.pdf* et *Linux – Les systèmes de fichiers*.
- Notions abordées : l'inode, le lien physique, l'affectation des droits.
- Commandes et fichiers exploités : `ln`, `mkdir`, `rm`, `stat`,
- Travail à rendre : Vous devrez répondre directement à plusieurs questions au sein de ce document. Vous le copierez sur Moodle sous le nom : « LAB3_NOM.pdf » **ainsi que les trois .c.**

2 Travaux préparatoires

Pour vous familiariser à nouveau avec la manipulation de fichiers :

- lisez et effectuez les commandes décrites aux chapitres §3.8 à §3.10 du support *Linux – les commandes.pdf*;

3 Manipulation de fichiers

3.1 Introduction

Pour manipuler les fichiers, on distingue les fonctions système dites de « bas niveau » (propre à chaque système d'exploitation) et les fonctions de la bibliothèque C standard dite de « haut niveau » commune à tous les systèmes. Nous ne voyons ici que quelques fonctions de bas niveau. Les fonctions de « haut niveau » ont été abordées lors de l'étude du langage C.

- Lisez le document *Support Linux – Les systèmes de fichiers*.

Sur votre système, quelle est la partition montée sur `/home` ? Quelle est la taille d'un bloc pour cette partition ?

```
df /home
pour mon systeme home est dans /dev/sda2
les bloc sont de 1K
```

Créez un fichier `new_file` contenant la date du jour (`echo 'date' > new_file`). Quels droits et quel numéro d'inode possède ce fichier ? Les droits attribués correspondent-ils à la valeur de l'UMASK ? **Justifiez.**

```
ls -lisa new_file
numero d'inode : 1446417
droit : -rw-rw-r--
umask
0002
les droits corresponde a la valeurs umask
car un fichier a les droit 666 et  $666 - 002 = 664$  ce qui est les droits du fichier new_file.
```

Créez un nouveau répertoire *new_rep*. Quelle commande utilisez-vous pour créer une copie de *new_file* dans ce répertoire au moyen d'un **lien physique** (nom du fichier copié : *copy_new_file*) ? Visualisez les attributs de ces deux fichiers (`ls -lisa new_file new_rep/copy_new_file`). Quels sont leurs numéros d'inode ? Quel est le nombre de liens ? Quel est le nombre de blocs utilisés ?

In `-lisa new_file new_rep/copy_new_file`

numero d'inode : 1446836 pour les deux

nombre de lien : 2 pour les deux.

Nombre de bloc utilisés : 8 pour les deux

Supprimez le fichier d'origine *new_file*. Quels sont les changements pour *copy_new_file* ? À quel moment l'inode relatif à ce fichier sera rendu au système ? Dans ce cas, les données constitutives du fichier seront-elles supprimées du support ? Quelle commande permettrait de s'assurer que les données d'un fichier supprimé sont devenues illisibles sur le support (sous Linux ? Sous Windows ?).

Le fichier n'a plus que 1 lien.

Lorsqu'il n'y aura plus de lien rattacher.

Non elle ne seront pas supprimer du support

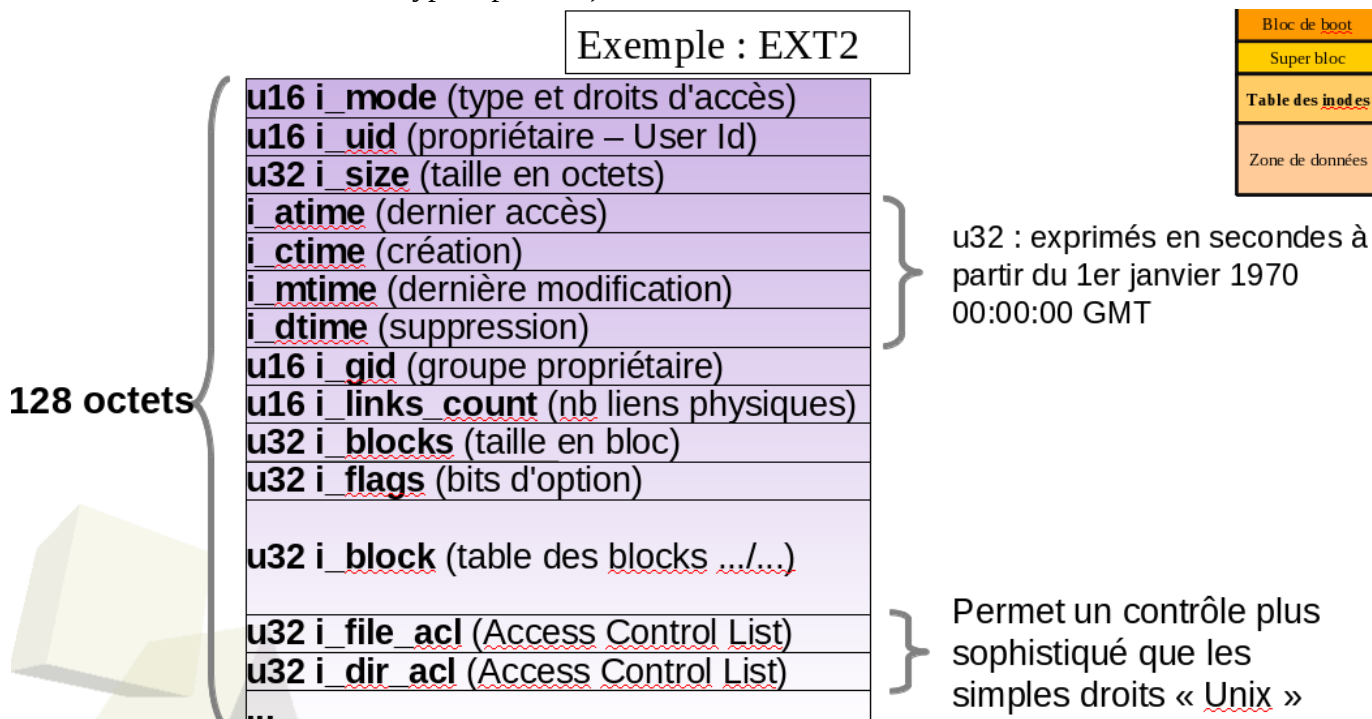
sous linux : `shred`

sous windows : `sdelete`

- Recréez *new_file* au moyen d'un lien physique

3.2 Accès aux informations de l'inode

Un inode est une entrée de la table des inodes. Cette entrée est constituée d'une structure nommée *stat* qui contient toutes les informations liées à l'inode (à l'exclusion des noms de fichier associés qui sont référencés dans les fichiers de type répertoire).



La table des blocs contient des pointeurs vers des zones du disque stockant le contenu du fichier.

En Shell :

La commande stat (man stat), du même nom que la structure, permet d'afficher les informations sur un inode (mise à par le contenu des blocs de données).

En C :

La fonction stat (man 2 stat) permet de récupérer les informations d'un inode dans une structure de type stat. La fonction stat est déclarée dans le fichier header unistd.h. La structure stat est déclarée dans le fichier header sys/stat.h.

```
#include <sys/stat.h>
#include <unistd.h>
int stat (char *nom, struct stat *buffer);
```

Explication des paramètres :

- char *nom : nom du fichier
- struct stat *buffer : pointeur vers une variable de type stat qui sera rempli lors de l'appel de la fonction.

Créez un programme en C qui récupère les informations de l'inode d'un fichier (ou d'un répertoire) que vous passez en paramètre. Pour voir ces informations, vous utiliserez l'outil de suivi des variables du débogueur DDD. Faites une copie d'écran montrant la variable argv[1] et les informations de l'inode correspondant au fichier passé en paramètre.

The screenshot shows the DDD debugger interface. The first window, titled '1: buffer', displays the contents of a struct stat. The fields are listed on the left, and their values are on the right. Some fields have sub-structures, like 'st_atim', 'st_mtim', and 'st_ctim', which are expanded to show 'tv_sec' and 'tv_nsec' values. The second window, titled '2: nom', shows the value '0x7fffffff702 "new_file"'. The background is a dotted grid.

Field	Value
st_dev	= 2050
st_ino	= 1446836
st_nlink	= 2
st_mode	= 33204
st_uid	= 1000
st_gid	= 1000
__pad0	= 0
st_rdev	= 0
st_size	= 33
st_blksize	= 4096
st_blocks	= 8
st_atim	= tv_sec = 1694524511 tv_nsec = 377083702
st_mtim	= tv_sec = 1694524360 tv_nsec = 332536019
st_ctim	= tv_sec = 1694526042 tv_nsec = 966167783
__glibc_reserved	= 0 0 0

2: nom
0x7fffffff702 "new_file"

Intégrez à votre programme une fonction qui convertit le temps Unix en une date exploitable par « Mme Michu ». Faites une copie d'écran de votre programme qui affiche le numéro d'inode et la date de création (ctime) du fichier passé en paramètre (**rendez le code C sur Moodle dans un fichier appelé part-3-2_NOM.c**).

```
victor@victor-G5-5590:~/TP/A4S7/systeme/TD3$ ./part-3-2_BOUVIER_D_ACHER new_file
numero inode: 1446836
date de création du fichier : Tue Sep 12 15:12:40 2023
```

« . » et « .. » sont des répertoires comme les autres.

Quels sont les numéros d'inode de « /home », « /home/new_rep/.. », « / », « /. » et « /.. » ?

Que constatez-vous ? Expliquez.

On constate que le repertoire . à le meme inode que le repertoire courant.

Le . est le lien du repertoire courant.

On constate que le repertoire .. à le meme inode que le repertoire parent.

Le .. est le lien du repertoire parent.

3.3 Fonctions de manipulation

La manipulation de fichiers consiste :

- soit à en modifier le contenu via un outil d'édition adapté au format du fichier (suite bureautique, éditeur de texte, logiciel de traitement d'images, etc.) ;
- soit à modifier les champs de leur inode (chown, chmod, touch, etc.) ;
- soit à modifier un fichier repertoire les référençant (cp, ln, mv, rm, etc.).

En C standard, les fonctions fopen, fclose, fread, fwrite, etc. permettent diverses manipulations indépendamment du système de fichiers (File System) sous-jacent.

En C système, les fonctions de manipulation permettent d'interagir directement sur les champs de l'inode ou dans les fichiers repertoire.

À titre d'exemple, nous allons voir comment modifier les droits d'un fichier.

En Shell :

Les commandes chmod, chown et chgrp permettent de modifier les attributs d'un fichier (droits et propriétaires).

Les utilisateurs et les groupes d'utilisateurs sur Linux sont en fait des nombres entiers (« User ID » et « Group ID ») que le système associe aux noms réels via les fichiers « /etc/passwd » pour l'UID et « /etc/group » pour le GID.

En Shell :

- Listez les groupes auxquels vous appartenez
- Créez deux fichiers « new_file » et « new_file2 »
- Changez le groupe propriétaire du fichier « new_file » (un parmi ceux auxquels vous appartenez)

Affichez les droits de « new_file » et « new_file2 »

groups

victor adm dialout cdrom sudo dip plugdev lpadmin lxd sambashare

chown :sudo new_fic

1447242 0 -rw-rw-r-- 1 victor sudo 0 sept. 16 23:50 new_fic

1451290 0 -rw-rw-r-- 1 victor victor 0 sept. 16 23:50 new_fic2

En C :

1 Autorisation d'accès

Avant de tenter toute manipulation de fichier par un programme (ou processus), il est nécessaire de savoir si le processus en question a le droit de le faire. Rappel : sous Linux un processus hérite des droits de l'utilisateur qui le lance.

La fonction `access` (man 2 `access`) permet de vérifier quels droits un processus possède sur un fichier.

```
#include <unistd.h>
int access (char *nom, int droit);
```

Explication des paramètres :

- `char *nom` : nom du fichier ;
- `int droit` : prends les valeurs des 4 constantes suivantes (prédéfinies dans le fichier header) :
 - `F_OK` : le fichier existe
 - `R_OK` : je peux lire le fichier
 - `W_OK` : je peux écrire dans le fichier
 - `X_OK` : je peux exécuter le fichier
- la valeur de retour est nulle si OK, sinon elle vaut -1 et la variable `errno` est renseignée.

2 Modification des attributs

Les fonctions `chmod()` et `fchmod()` permettent de modifier les droits d'un fichier :

- `chmod()` modifie les droits d'un fichier par son nom;
- `fchmod()` modifie les droits d'un fichier par son descripteur renvoyé par la fonction `open()` (man 2 `open`).

```
#include <sys/stat.h>
int chmod (char *nom, int droits);
int fchmod (int file, int droits);
```

Explication des paramètres :

- `char *nom` : nom du fichier ;
- `int file` : descripteur de fichier ouvert au préalable par la fonction `open()` ;
- `int droits` : droits attribués au fichier. Utilisez au choix une valeur octale ou une association de bits comme dans la structure `stat`.

Les fonctions `chown()` et `fchown()` permettent de modifier le « propriétaire » et le « groupe propriétaire » d'un fichier (`uid` et `gid`)

- `chown()` : modifie `uid` et `gid` d'un fichier par son nom;
- `fchown()` : modifie `uid` et `gid` d'un fichier par son descripteur renvoyé par la fonction `open()`.

```
#include <sys/unistd.h>
int chown (char *nom, int uid, int gid);
int fchown (int file, int uid, int gid);
```

Explication des paramètres :

- `char *nom` : nom du fichier;
- `int file` : descripteur de fichier ouvert au préalable par la fonction `open()`;
- `int uid` : numéro du futur propriétaire du fichier;
- `int gid` : numéro du futur groupe propriétaire du fichier.

Remarque : Si on ne désire modifier qu'un des deux paramètres (`uid` ou `gid`), il suffit de renseigner le deuxième avec la valeur spéciale -1.

- Réalisez un programme en C qui affiche les droits que votre processus possède sur un fichier (ou un répertoire) que vous passez en paramètre (**rendez le code C sur Moodle dans un fichier appelé `part-3_3_NOM.c`**).

- Copiez ici une capture d'écran de son exécution.

```
victor@victor-G5-5590:~/TP/A4S7/systeme/TD3$ ./part-3-3_BOUVIER_D_ACHER part-3-2_BOUVIER_D_ACHER
le processus peut lire le fichier
le processus peut ecrire le fichier
le processus peut executer le fichier
```

La suite de ce document fournit les fonctions système permettant toutes les manipulations sur les fichiers. La commande « man 3 nom_fonction » fournit l'aide nécessaire à l'exploitation de ces fonctions.

3 Création/suppression

- `int creat(char *nom, int droits)` : création d'un fichier « classique ». On préfère de plus en plus utiliser la fonction `open()` décrite dans la rubrique suivante.
- `int mkdir(char *nom, int droits)` : création d'un fichier « répertoire ».
- `int mkfifo(char *nom, int droits)` : création d'un fichier « pipe ».
- `int mknod(char *nom, int droits, int dev)` : création d'un fichier périphérique.
- `int link(char *source, char *cible)` : création d'un lien physique vers un fichier.
- `int symlink(char *source, char *cible)` : création d'un lien symbolique vers un fichier.
- `int unlink(char *nom)` : décrémente le nombre de liens physiques d'un fichier, si le compteur de lien atteint zéro, l'inode est rendu au système.
- `int rmdir(char *nom)` : supprime un répertoire à condition que ce dernier soit vide.

4 Accès au contenu

- `int open (char *nom, int mode[int droits])` : demande l'allocation d'une nouvelle entrée dans la table des fichiers ouverts du système. La fonction renvoie un entier « descripteur de fichier » pour le fichier dont le nom est passé en paramètre. Ce descripteur servira de référence ultérieure pour accéder au fichier et pour le fermer. Cette fonction permet l'ouverture d'un fichier existant ou la création d'un nouveau fichier (évite d'utiliser la fonction `creat()`). `int mode` peut prendre les valeurs des constantes prédéfinies suivantes :
 - `O_RDONLY` : ouverture en lecture seule;
 - `O_WRONLY` : ouverture en écriture seule;
 - `O_RDWR` : ouverture en lecture-écriture.

Et facultativement des constantes prédéfinies suivantes :

- `O_CREAT` : création du fichier s'il n'existe pas;
- `O_TRUNC` : vidage préalable du fichier;
- `O_APPEND` : écriture à la fin du fichier;
- `O_EXCL` : ne pas ouvrir le fichier s'il n'existe pas.

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
void main(void) {
    int desc;
    if ((desc=open("test.txt", O_RDWR|O_CREAT|O_EXCL, 0777)) < 0)
        perror("erreur d'ouverture");
    else close(desc);
}
```

- `int close (int desc)` : ferme un fichier préalablement ouvert par `open()`;
- `int read (int desc, char *buffer, int nb)` : lis un volume d'octets dans un fichier ouvert avec `open()`;
- `int write (int desc, char *buffer, int nb)` : écris un volume d'octets dans un fichier

- ouvert avec `open()`;
- `off_t lseek (int desc, off_t offset, int whence)` : déplace le pointeur interne de lecture/écriture.

- Copiez sur Moodle sous le nom « part-4_NOM.c »**, un programme en C qui crée un nouveau fichier dont le nom est passé en paramètre. Ce fichier doit être en « lecture seule » et uniquement pour son propriétaire. Ce fichier contient au format XML (cf. exemple ci-dessous) la date et l'heure actuelle, le nom de l'utilisateur, son UID et un mot de passe aléatoire de 10 caractères que vous générez.

Le format XML est un format texte utilisant des balises que vous définissez, par exemple :

```
<nouvel_usager>
    <jour>yyyyyyyy</jour>
    <nom>xxxxxxx</nom>
    etc.
</nouvel_usager>
```

Pour info, le HTML est un format XML dont les balises sont normalisées.

Vous devez respecter les règles suivantes quant au choix de vos balises : Elles sont en anglais. Elles sont préfixées par **vos** initiales (exemple : `<AB_date> ... </AB_date>`).

- Lancez votre programme en tant qu'utilisateur sans privilège (non root). Faites une copie d'écran montrant les droits du fichier généré et son contenu.

```
victor@victor-G5-5590:~/TP/A4S7/systeme/TD3$ ./part-4_BOUVIER_D_ACHER test
victor@victor-G5-5590:~/TP/A4S7/systeme/TD3$ ls -lisa test.xml
1451514 4 -r----- 1 victor victor 195 sept. 17 01:38 test.xml
victor@victor-G5-5590:~/TP/A4S7/systeme/TD3$ cat test.xml
<VBdA_user>
    <VBdA_date>
        dim. 17 sept. 2023 01:38:36 CEST
    </VBdA_date>
    <VBdA_owner>
        victor
    </VBdA_owner>
    <VBdA_uid>
        1000
    </VBdA_uid>
    <VBdA_mdp>
        .EKh3E:rR4
    </VBdA_mdp>
</VBdA_user>
```